

Passing Networks Studio

The code, explained

This bundle is a FastAPI server (api_server.py) plus a browser UI that runs the whole passing-network pipeline on real match data. The heavy lifting - all fourteen analysis modules - lives in the course engine, predict_server.py, which the server reuses unchanged. This document explains api_server.py in full, then tours what each engine module computes.

1. Setup and the JSON safety net

The imports, then os.chdir() so the server finds its HTML/CSS/JS and the Excel datasets no matter where you launch python from. predict_server is imported here; that is safe because all of its work sits behind an 'if __name__ == "__main__"' guard, so nothing runs until we call load_data(). _clean() walks the engine's output and turns any NaN or Infinity float into null - those are not valid JSON tokens and would otherwise break the browser's JSON.parse.

```
"""
Build and Analyse Passing Networks - standalone studio.

A FastAPI server + browser UI that runs the whole analysis pipeline on real
StatsBomb match data: passing networks, centrality algorithms, tactical maps
(25-zone heatmaps, sonars, zone flow), the match timeline and the verdict.

It wraps the course engine (predict_server.py) unchanged - that single file
holds all fourteen analysis modules (network construction, degree /
betweenness / eigenvector / clustering centrality, Grund centralisation,
weakest link, PPDA, field tilt, team shape, pass sequences, timeline and
momentum, and the match verdict). This server just loads the data once and
exposes those modules over HTTP for the browser UI.

Run:
    pip install -r requirements.txt
    python api_server.py
    # open http://localhost:8000
"""

import os
import sys
import json
import re
import time
import io
import base64

# Serve the HTML/CSS/JS and the Excel datasets by relative path regardless of
# where the server is launched from.
os.chdir(os.path.dirname(os.path.abspath(__file__)))

from contextlib import asynccontextmanager, redirect_stdout

from fastapi import FastAPI, Request
from fastapi.responses import FileResponse, Response
import uvicorn
import pandas as pd

# The engine. Importing it is safe: all of its work sits behind a
# `if __name__ == "__main__"` guard, so nothing runs until we call load_data().
import predict_server as engine

def _clean(o):
    """Recursively turn NaN / Infinity floats into null so the JSON is valid
    for the browser's JSON.parse (NaN/Infinity are not legal JSON tokens)."""
    if isinstance(o, float):
        return None if (o != o or o == float("inf") or o == float("-inf")) else o
    if isinstance(o, dict):
```

```

    return {k: _clean(v) for k, v in o.items()}
if isinstance(o, list):
    return [_clean(v) for v in o]
return o

```

```
URL = "http://localhost:8000"
```

```

@asynccontextmanager
async def lifespan(app):
    n = engine.load_data()
    bar = "=" * 56

```

2. Startup, static files, and the one endpoint

The lifespan hook loads all 31 matches once at startup (this takes ~20 seconds - it reads two Excel files) and prints the ready banner. Three routes serve the UI files. The whole pipeline is exposed through a single endpoint: the browser POSTs a command such as `{command:'network', match_date:'2019-06-01', team:'Liverpool'}` and gets that module's result. `engine.handle_command()` routes it to the right handler; we redirect the engine's own prints to `stderr` so the HTTP response is clean JSON, then clean and return it. Finally `uvicorn` serves it on port 8000.

```

print(f"\n {bar}\n Passing Networks studio is ready ({n} matches).\n"
      f" Open {URL} in your browser.\n {bar}\n", flush=True)
yield

```

```
app = FastAPI(title="Passing Networks", lifespan=lifespan)
```

```

@app.get("/")
def index():
    return FileResponse("main.html")

```

```

@app.get("/style.css")
def css():
    return FileResponse("style.css", media_type="text/css")

```

```

@app.get("/script.js")
def js():
    return FileResponse("script.js", media_type="application/javascript")

```

```

@app.post("/api/predict")
async def predict(req: Request):
    """One endpoint for the whole pipeline. The browser posts a command such as
    {command: 'network', match_date: '2019-06-01', team: 'Liverpool'} and gets
    that module's result. Valid commands: matches, teams, network, insights,
    tactical, tactical_map, zone_directions, zone_connections, timeline, goals,
    shots, compare, player, health."""
    cmd = await req.json()
    # Route the engine's own prints to stderr so the HTTP response is clean JSON.
    with redirect_stdout(sys.stderr):
        result = engine.handle_command(cmd)
    body = json.dumps(_clean(result), default=str)
    return Response(content=body, media_type="application/json")

```

```

def _json(obj):
    return Response(json.dumps(_clean(obj), default=str), media_type="application/json")

```

```

def _register(df, key_hint):
    """Validate one match dataframe and register it in the engine's in-memory
    store under a unique key. Returns a summary, or None if the schema is wrong."""
    if {"team_name", "event_type", "minute"} - set(df.columns):
        return None

```

```

if "xt_added" not in df.columns:
    df["xt_added"] = 0.0
key = re.sub(r"^[A-Za-z0-9_\.]+\.", "_", str(key_hint)).strip("_") or ("upload_%d" % int(time.time()))
base, i = key, 1
while key in engine.MATCH_DATA:
    i += 1
    key = f"{base}_{i}"
engine.MATCH_DATA[key] = df
return {"match_date": key, "events": int(len(df)),
        "teams": df["team_name"].dropna().unique().tolist()}

@app.post("/api/upload")
async def upload(req: Request):
    """Add match(es). The browser sends either a single CSV as text, or a
    multi-sheet Excel workbook as base64 - and every valid sheet in the
    workbook becomes its own match in the dropdown. New matches show up in the
    match list exactly like the built-ins."""
    body = await req.json()

    # --- Excel workbook: register EVERY sheet as its own match ---
    b64 = body.get("xlsx_b64")
    if b64:
        try:
            xl = pd.ExcelFile(io.BytesIO(base64.b64decode(b64)))
        except Exception as e:
            return _json({"error": f"Could not read the Excel file: {e}"})
        return _register_workbook(xl)

    # --- Single CSV ---
    csv_text = body.get("csv_text") or ""
    if not csv_text.strip():
        return _json({"error": "No CSV or Excel content received."})
    name = (body.get("match_name") or "").strip()
    if name.lower().endswith(".csv"):
        name = name[:-4]
    info = _register(pd.read_csv(io.StringIO(csv_text)), name)
    if not info:
        return _json({"error": "CSV is missing required columns (team_name, event_type, minute)."})
    return _json({"status": "ok", "added": [info], "matches_loaded": len(engine.MATCH_DATA)})

def _register_workbook(xl):
    added = []
    for sheet in xl.sheet_names:
        if str(sheet).strip().lower() == "summary":
            continue
        info = _register(pd.read_excel(xl, sheet_name=sheet), sheet)
        if info:
            added.append(info)
    if not added:
        return _json({"error": "No valid match sheets found (each needs team_name, event_type, minute)."})
    return _json({"status": "ok", "added": added, "matches_loaded": len(engine.MATCH_DATA)})

if __name__ == "__main__":
    # log_level="warning" keeps the console to just the ready banner. The server
    # binds 0.0.0.0 (reachable on your network); open it at http://localhost:8000.
    uvicorn.run(app, host="0.0.0.0", port=8000, log_level="warning")

```

3. The engine: the fourteen modules

api_server.py is deliberately thin - it is a doorway to predict_server.py, whose COMMAND_HANDLERS dictionary maps each command to a handler. Here is what each one computes. You never run these by hand; the server calls them and the UI draws the result.

network	build the pass matrix (directed passer->receiver counts) and average positions; edges weighted by passes. metrics: density = edges/(n(n-1)), hub. (unique out + unique in connections) / (2(n-1)).
degree	distance = 1/passes; all-pairs shortest paths; share running through a player.
betweenness	power iteration $c = A.c$, normalised each round (the principal eigenvector).
eigenvector	edges among a player's k neighbours / (k(k-1)/2).
clustering	0.40 involvement-without-support + 0.30 pressability + 0.30 xT-bottleneck, then x a role weight (CB/GK 1.3, CM 1.2, FB/W 1.0, ST 0.8).
weakest link	Grund centralisation = sum(hub - player involvement) / (n x total passes); plus playing style and a possession estimate.
tactical	25-zone (5x5) heatmap; zone = row x 5 + col (col = x/24, row = y/16).
tactical_map	per-zone pass angles binned - the direction sonar.
zone_directions	progressive / regressive / lateral zone-to-zone flow + busiest corridors.
zone_connections	both teams side by side, incl. PPDA and field tilt:
compare	opp passes in own half / your tackles+interceptions+fouls+blocks in their half.
PPDA	your final-third passes / both teams' final-third passes.
field tilt	eighteen-plus 5-minute windows (passes, possession, xT); momentum shifts, dominant periods; and the match verdict.
timeline	justice = 25 + 18.75 x (metrics the winner led: xG / xT / possession / passes).
verdict	trace back from each goal; build-up xT = sum of positive delta xT (12x8 grid).
goals	per player: shots, goals, xG, conversion rate, xG difference (goals - xG).
shots	

4. The data

Two Excel workbooks ship with the bundle: champions_league_finals_with_xt.xlsx (the 2018 and 2019 CL finals) and additional_matches_with_xt.xlsx (29 more matches from Euro 2024, World Cup 2022, Copa America 2024, the 2023 Women's World Cup and AFCON 2023). Each sheet is one match of StatsBomb event data - one row per action - and every action already carries an xt column, so the threat maths is precomputed. The engine loads every sheet into memory keyed by match date.